

WEEK 4

DUE: FOLLOWING SATURDAY

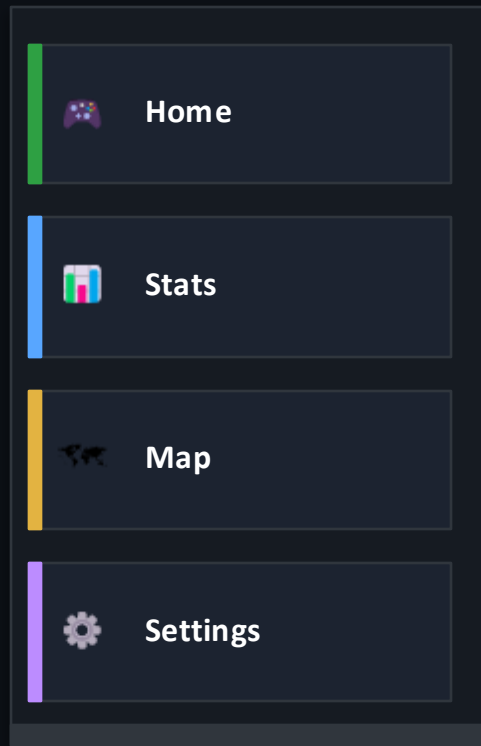
MARKED

The Real App

Three modes is a portfolio.

Three modes inside a structured, polished app is a project.

Wrap everything into a proper app shell with real platform features.



Project Structure

Refactor into folders first. This is the most important task of the week.

PlayHubApp /

App/

PlayHubApp.swift

Models/

GameMode.swift

GameSession.swift

TriviaQuestion.swift

ViewModels/

TapFrenzyVM.swift

LightItUpVM.swift

QuizRushVM.swift

StatsVM.swift

Services/

TriviaAPI.swift

NotificationService.swift

LocationService.swift

Views/

Tabs/ HomeTab.swift

StatsTab.swift

MapTab.swift

SettingsTab.swift

Games/ TapFrenzyView.swift

LightItUpView.swift

QuizRushView.swift

Shared/

ResultView.swift

ScoreBadge.swift

Why this structure?

- Easier to navigate & • Each file has one job
- Week 5 walkthrough-ready

GameSession

id: UUID

mode: GameMode

score: Int

timestamp: Date

latitude: Double

longitude: Double

Every completed game

appends one session.

Persisted as JSON

in UserDefaults.

What to Build

Tab Bar Shell



TabView with 4 tabs (Home, Stats, Map, Settings). SF Symbols for icons. Each tab is its own UINavigationController.

Stats Screen



Every completed game writes a GameSession. Stats tab shows totals, personal bests, recent games, and a bar chart per mode using the Charts framework.

Map of Games



MapKit + Core Location. Each completed session records the current location. Map tab shows pins — tap a pin to see that session's score.

Daily Challenge



UserNotifications. Settings lets the player pick a time. A daily local notification is scheduled at that time.

Share Your Score



ShareLink on the Result screen. Generates a share string like 'I just scored 47 on Quiz Rush — beat that!'. One line of SwiftUI.

Settings



Notifications toggle, daily challenge time picker, and a reset-all-stats button with a confirmation dialog.

How to Approach It

1

Refactor Folders First

Move existing code into the folder structure. Don't bolt on features. This is the most important step — clean structure makes everything else easier.

2

Add the Tab Bar

Replace root `NavigationStack` with `TabView`. Each tab is a `NavigationStack`. Use SF Symbols: `gamecontroller`, `chart.bar`, `map`, `gear`.

3

GameSession + Persist

Struct: `id`, `mode`, `score`, `timestamp`, `latitude`, `longitude`.
Save array to `UserDefaults` as encoded JSON. Append on every game end.

4

Build Stats Tab

Read sessions. Show totals, bests, recent games list. Add bar chart per mode using SwiftUI Charts — one `BarMark` per session.

5

Add Map Tab

Wrap `CLLocationManager` in `LocationService`. Request permission on launch.
Record location on each session. Display pins using Map with `Marker`.

6

Wire Notifications

`NotificationService` wraps `UNUserNotificationCenter`. Request permission.
Schedule a daily repeating notification at the user's chosen time.

7

ShareLink on Result

One `ShareLink` view on the result screen. Pass a formatted string.
The system handles the share sheet — one line of SwiftUI.

Submission Checklist

DUE FOLLOWING SATURDAY — FINAL MARKED SUBMISSION



Project restructured into clear folders (Models, Views, ViewModels, Services)



TabView shell with at least 4 tabs working



Stats tab aggregates results across all three game modes



Map tab shows pins for completed sessions using Core Location



Settings tab schedules a daily local notification at a chosen time



ShareLink works on the Result screen



All three previous game modes still work — nothing regressed



README: architecture overview, features list, known limitations, & Reflection



Final walkthrough — every line is yours to explain